

1. Introduction:

In this project, the aim was to develop *StockSim*, a market analysis and trading strategy simulation tool using historical data developed fully in C++. The simulator is an object-oriented program with custom implementations of basic data structures such as linked lists, stacks, queues, and binary search trees. It uses historical data from the stock market to evaluate numerous investment strategies across two decades and assess their performance.

The main research question posed in this project is:

Does a Dynamic Systematic Investment Plan (Dynamic SIP) outperform a traditional Fixed SIP over long historical market periods?

This research question is critical since many investors use dollar-cost averaging schemes for managing risk in their investments. While a Fixed SIP invests a fixed amount of cash at certain intervals irrespective of market trends, a Dynamic SIP is a method in which investment strategies are dynamically modified according to price trends or volatility in the market. Thus, by evaluating both the strategies during different market trends such as the dot-com crash, the 2008 financial crisis, and the 2010–2020 bull market, we can determine whether or not dynamic investment behavior pays off using the simulator.

The simulator uses historical daily price data obtained from Yahoo Finance CSV files. Three datasets were included in the project:

- SPX (S&P 500 Index)
- AMZN (Amazon)
- NVDA (NVIDIA)

The data in each set included open, high, low, closing, adjusted closing prices, and volumes for each day. All analysis was limited to January 1, 2000, through January 1, 2020, to provide a basis for comparison among all the strategies. As the stock market does not operate on weekends and public holidays, the simulator was coded to account for such discontinuities without the assumption of consecutive days.

It is noteworthy that one of the purposes of the current project was the implementation of data structures in an authentic setting. The linked list stored historical price information, while the queue served as a tool to calculate the moving average and pending orders. Meanwhile, the stack was used to keep the trade history record for the undo feature. Finally, the binary search tree sorted the stocks according to the return criteria.

2. Data Structure Design

This section describes the core data structures used in the *StockSim* trading system. The system simulates a financial market environment with historical price tracking, order processing, trading strategies, and performance-based stock organization. Each data structure is selected based on realistic trading requirements and a detailed analysis of time complexity.

2.1 Doubly Linked List — PriceHistory

The PriceHistory class was implemented as a doubly linked list that stores daily historical stock prices. Each node represents one trading day and contains the date, open, high, low, close, volume, and pointers to the next and previous nodes.

A doubly linked list was chosen because stock price data is inherently sequential and time-ordered. New data is continuously appended as new trading days occur, making append operations more frequent than searches.

The structure also supports bidirectional traversal. Forward traversal is used for chronological simulation, while reverse traversal is essential for indicators such as momentum, moving averages, and trailing returns.

Time Complexity:

Operation	Complexity	Justification
Append	$O(1)$	Because a tail pointer is maintained, insertion does not require traversal of the list. The new node is directly linked to the last node.
Find by date	$O(n)$	In the worst case, the list must be scanned sequentially from head to tail since nodes are not indexed or sorted for direct access.

Forward Traversal	$O(n)$	Every node must be visited once in sequence.
Reverse Traversal	$O(n)$	Each node is visited once using prev pointers; no random access exists.

Why Linked Lists Were Appropriate:

The linked list structure was ideal for historical stock data because:

- Stock data arrives sequentially and is only appended
- No resizing or shifting is required (unlike arrays)
- Memory allocation is dynamic, allowing unlimited growth
- Reverse traversal is required for technical analysis

The $O(1)$ append complexity is achieved because the system stores a direct reference to the tail node, eliminating traversal cost. The $O(n)$ search complexity is acceptable because queries are infrequent compared to appends and traversals.

2.2 Circular Queue — Moving Average Computation

The CircularQueue class was implemented as a fixed-size circular buffer used for moving average calculations. This structure played a critical role in the Golden Cross trading strategy. Moving averages require maintaining a rolling window of recent stock prices. A circular queue is highly efficient for this task because old values can be overwritten automatically once the queue becomes full.

The queue maintained:

- A dynamically allocated array
- Head index
- Tail index
- Current element count
- Fixed capacity

The enqueue operation inserts a new value at the tail position. If the queue is already full, the oldest value is overwritten automatically by advancing the head pointer. This design avoids expensive shifting operations that would occur in a standard array implementation.

Time Complexity:

Operation	Complexity	Justification
Enqueue	$O(1)$	Insertion is done at the tail index without shifting elements. Index arithmetic handles wrap-around using modulo operations.
Dequeue	$O(1)$	Removal is performed by moving the head pointer forward; no shifting of elements is required.
Peek	$O(1)$	Accessing front element is a direct array index operation.
getAverage	$O(n)$	All elements in the current window must be summed to compute the average.

Why Circular Queues Were Appropriate:

The circular queue was the best choice because:

- Moving averages require fixed-size rolling windows
- Constant-time insertion and removal are critical
- Memory usage remains constant
- Overwriting old data simplifies implementation

The Golden Cross strategy specifically used:

- 50-day moving average queue
- 200-day moving average queue

These rolling averages were updated daily during historical simulation.

2.3 TradeStack — Undo Functionality

The TradeStack class was implemented as a pointer-based stack used to store trade history and support undo functionality.

Each stack node contained a TradeRecord structure storing:

- Ticker symbol
- Trade date
- Price
- Number of shares
- Buy or sell action
- Total trade cost

Stacks follow Last-In-First-Out (LIFO) behavior, making them ideal for undo systems because the most recent action should always be reversed first.

When a user executed a trade, the trade record was pushed onto the stack. If the user selected the undo option, the top trade was popped and reversed automatically.

Time Complexity:

Operation	Complexity	Justification
Push	$O(1)$	New nodes are inserted at the top of the stack without traversal.
Pop	$O(1)$	The top pointer is moved to the next node directly.
Peek	$O(1)$	The top element is directly accessible.
isEmpty	$O(1)$	A simple null pointer check is performed.

Why Stacks Were Appropriate:

The stack structure was ideal because:

- Undo systems naturally require LIFO behavior

- Recent trades are accessed most frequently
- Constant-time insertion and removal improve efficiency
- Pointer-based implementation allows dynamic growth

The $O(1)$ complexity for push and pop is achieved because only pointer adjustments are required, with no traversal or shifting.

2.4 OrderQueue — Pending Order Management

The OrderQueue class was implemented as a node-based FIFO queue for pending trade orders.

Each order contained:

- Ticker symbol
- Order type
- Buy or sell side
- Target price
- Share quantity
- Submission date

Queues follow First-In-First-Out (FIFO) behavior, which accurately models how pending orders are processed in many real-world systems.

New orders were inserted at the rear of the queue, while execution always occurred from the front.

Time Complexity:

Operation	Complexity	Justification
Enqueue	$O(1)$	New nodes are inserted at the rear using a tail pointer.
Dequeue	$O(1)$	Removal is performed at the front without traversal.
Peek	$O(1)$	Front node is directly accessible.
isEmpty	$O(1)$	Null pointer check.

Why Queues Were Appropriate:

The queue structure was ideal because:

- Orders must be processed in chronological order
- FIFO behavior matches real trading systems
- Constant-time operations improve efficiency
- Dynamic node allocation avoids capacity limitations

The $O(1)$ complexity is achieved through direct front and rear pointer manipulation.

2.5 Binary Search Tree — Stock Performance Indexing

The StockBST class was implemented as a binary search tree used to organize stocks by annual return percentage.

Each BST node stored:

- Ticker symbol
- Performance key
- Year
- Left child pointer
- Right child pointer

The BST allowed stocks to be sorted automatically by return percentage.

Inorder traversal displayed stocks in ascending order of return performance. Range search operations allowed users to query stocks whose returns fell within specified intervals.

Time Complexities:

Operation	Complexity	Justification
Insert	$O(\log n)$	In a balanced tree, each comparison eliminates half of the remaining nodes. In a skewed tree, it degenerates into a linked list.

Search	$O(\log n)$	Same reasoning as insertion; tree height determines performance.
Range Search	$O(n)$	All nodes may need to be visited to collect values in range.
Traversal	$O(n)$	Each node is visited exactly once.

Worst-case complexity could degrade to $O(n)$ if the tree became unbalanced.

Why BSTs Were Appropriate:

BSTs were useful because:

- Financial data often requires sorted ranking
- Range queries are common in performance analysis
- Recursive traversal simplifies implementation
- Dynamic insertion allows flexible indexing

The $O(\log n)$ average complexity occurs because each comparison reduces the search space by approximately half, assuming a balanced tree.

2.6 Templates — StockManager

The StockManager class was implemented as a generic template class capable of managing collections of FinancialAsset-derived objects.

The template design allowed the same manager to work with both:

- Stock objects
- ETF objects

without duplicating code.

The class supported:

- Adding assets
- Removing assets

- Searching by ticker
- Sorting by annual return
- Sorting alphabetically

Time Complexity:

Operation	Complexity	Justification
Add Asset	$O(1)$ or $O(\log n)$	Depends on underlying container (vector or tree).
Remove Asset	$O(n)$	Requires searching for the asset first.
Search	$O(n)$	Linear scan unless indexed structure is used.
Sort	$O(n \log n)$	Standard comparison-based sorting algorithm.

Why Templates Were Appropriate:

Templates improved the design because:

- Code duplication was minimized
- The manager remained flexible and reusable
- Generic programming improved scalability
- Polymorphism integrated naturally with inheritance

Sorting complexity follows $O(n \log n)$ due to comparison-based sorting algorithms such as merge sort or quicksort used internally.

AI Usage — Section 2: StockManager

Prompt: "How to implement a template class for the different objects if sharing similar methods?"

AI Output: To implement a template class for objects sharing similar methods, you define the class using a placeholder type, typically template <typename T>. This allows the StockManager to handle any object—be it a Stock or an ETF—as long as that object provides the specific member functions the manager calls.

Modifications: None

3. Trading Strategies

This section explains each of the four trading strategies implemented in the *StockSim* trading system.

3.1 Fixed SIP

The Fixed SIP strategy invested a constant dollar amount on the first trading day of every month regardless of market conditions.

This strategy represents traditional long-term passive investing behavior commonly used in retirement accounts and index investing.

The algorithm operated as follows:

1. At the start of each month, a fixed contribution was added.
2. The entire monthly amount was invested immediately.
3. Fractional shares were purchased at the closing price.
4. Shares accumulated continuously throughout the 20-year simulation.

The Fixed SIP strategy served as the baseline benchmark for comparing all other strategies.

Expected Behavior:

The expected advantages of this strategy included:

- Simplicity
- Consistent investing discipline
- Reduced emotional trading
- Long-term compounding benefits

However, the strategy does not adapt to market conditions and therefore continues investing identical amounts even during expensive market periods.

3.2 Dynamic SIP

The Dynamic SIP strategy evolved beyond the Fixed SIP model by adapting investment amounts to multiple market condition indicators simultaneously. Rather than using a single moving average, the strategy employed dual moving averages, drawdown tracking, volatility measurement, and momentum confirmation to optimize capital deployment.

The strategy operated as follows:

1. A fixed monthly contribution was added to reserve cash.
2. The strategy compared a 50-day moving average (short-term trend) to a 200-day moving average (long-term trend) to determine bear/bull market conditions.
 - a. Bull market ($MA50 \geq MA200$): Base deployment of 12% of monthly capital
 - b. Bear market ($MA50 < MA200$): Base deployment of 90% of monthly capital
3. The primary signal for deployment sizing was the drawdown—the percentage decline from the peak price to current price.
 - a. Drawdown $< 5\%$: Reduce deployment to 35% of base
 - b. Drawdown 5–10%: Maintain base deployment (85%)
 - c. Drawdown 10–20%: Increase deployment to $2.30\times$ base
 - d. Drawdown 20–30%: Increase deployment to $3.60\times$ base
 - e. Drawdown $> 30\%$: Maximize deployment to $4.80\times$ base
4. The current price was compared to the 50-day moving average to refine deployment:
 - a. If price $< 96\%$ of MA50 (dip): Increase deployment by $1.80\times$
 - b. If price $> 102\%$ of MA50 (expensive): Reduce deployment to $0.45\times$
5. When high volatility, reduced aggressiveness to preserve reserve runway:
 - a. Volatility $> 3.0\%$: Reduce deployment by 10%
 - b. Volatility $> 2.0\%$: Reduce deployment by 5%
6. Aggressive buys ($> 2.50\times$ monthly capital) required one of three conditions to prevent false-bottom purchases:
 - a. Price recovered above MA50, or
 - b. Positive 5-day momentum, or
 - c. Deep drawdown ($> 15\%$) with near-zero or positive momentum.
7. The reserve cash was handled as such:

- a. Reserve cap forced mandatory deployment if idle cash exceeded 36 months of contributions
 - b. End-game convergence gradually deployed all remaining reserves in the final months to ensure full capital utilization.
8. Total long-term capital deployment remained equal to the Fixed SIP strategy.

Parameters Used:

Parameter	Value
Short-term Moving Average	50 days
Long-term Moving Average	200 days
Volatility Window	20 days
Momentum Window	5 days
Dip Threshold	96%
Expensive Threshold	102%
Reserve Cap	36 months

Expected Behavior:

The strategy was expected to:

- Perform significantly better during volatile or crash markets
- Accumulate maximum shares during deep drawdowns (20%+)
- Reduce purchases and build reserves during bull phases
- Improve long-term compounding through lower average cost basis
- Maintain full capital deployment without exceeding budget constraints

The Dynamic SIP strategy was the primary focus of the research analysis.

3.3 Golden Cross Strategy

The Golden Cross strategy was based on technical analysis using moving average crossovers.

The strategy maintained:

- A 50-day moving average
- A 200-day moving average

A buy signal occurred when the 50-day moving average crossed above the 200-day moving average. This event is commonly known as a Golden Cross and is interpreted as a bullish trend signal.

A sell signal occurred when the 50-day moving average crossed below the 200-day moving average. This event is known as a Death Cross and signals potential bearish market conditions.

The strategy operated as follows:

1. Daily closing prices were inserted into both circular queues.
2. Moving averages were updated continuously.
3. When a Golden Cross occurred, all available cash was invested.
4. When a Death Cross occurred, all shares were sold and converted to cash.

Expected Behavior:

The Golden Cross strategy was expected to:

- Capture long-term bullish trends
- Avoid severe bear market drawdowns
- Trade less frequently than short-term systems

However, the strategy may lag during rapid market reversals because moving averages are inherently delayed indicators.

3.4 Momentum Strategy

The Momentum Strategy used trailing 6-month returns to determine whether market momentum was positive or negative.

The strategy calculated the percentage return over approximately 126 trading days.

The algorithm operated as follows:

1. Monthly contributions were added to cash reserves.
2. Six-month trailing returns were computed.
3. If momentum exceeded a positive threshold, the strategy invested all available cash.
4. If momentum became negative, all positions were sold.

This strategy was based on the financial theory that assets with positive recent performance often continue performing well in the short to medium term.

Parameters Used

Parameter	Value
Moving Average Window	30 days
Dip Threshold	95%
Expensive Threshold	105%

Expected Behavior:

The Momentum Strategy was expected to:

- Follow strong market trends
- Avoid weak market periods
- Generate moderate trade frequency

However, the strategy may produce false signals during highly volatile sideways markets.

AI Usage — Section 3:

Prompt: “Explain the best way to maximize return on dynamic SIP for dummies”

AI Output: The Dynamic SIP strategy works by continuously adding monthly contributions to a cash reserve and adjusting investment size based on market conditions, buying more during dips and reducing exposure when prices are high or volatility increases. It uses moving averages and drawdown signals to determine whether the market is in a bullish or bearish regime, with more aggressive accumulation during downturns and partial cash preservation in overheated conditions.

Modification: Asked AI to implement some of these strategies, mainly aggressive buying conditionals.

4. Analysis:

This section will detail the various analyses and insights gained from the project.

4.1 Testing Method:

All four datasets were tested using the SPX dataset on a date range of 2000-2020.

4.2 Insights- Dynamic vs. SIP:

The first key insight drawn from the backtesting results is the fact that the Dynamic SIP strategy needed much more fine-tuning and optimization than the Fixed SIP strategy while only delivering slightly better results on average. The performance of the Dynamic SIP algorithm is highly dependent on the moving averages thresholds, the dip percentage, cash reserve dynamics, and the factors defining investment scaling, meaning that small parameter changes could have a significant impact on the overall performance across different market periods, which could be a reason to consider the strategy more sensitive and less robust than the Fixed SIP strategy.

This idea can be clearly observed by comparing the results delivered by the two tested algorithms. The Dynamic SIP strategy delivered the highest portfolio value of \$276,909 compared to \$276,886 delivered by the Fixed SIP strategy. Notably, there is an extremely insignificant gap of less than \$25 in favor of the former option after a twenty-year period of investments. The total returns of both algorithms were almost equal and stood at about 130.7%, whereas the CAGR rates were absolutely equal and equaled 4.27%.

In times of extreme volatility, especially the 2008 financial crisis, the Dynamic SIP approach showed superior performance since it would buy more shares in times of steep market decline, which is much better in a vacuum. The idea behind Dynamic SIP was to invest more in market downturns below the moving average threshold values. More investments resulted in obtaining more discounted shares and participating more in the subsequent price rise. But the performance gain was not big enough to make up for the extra complexity of implementation, namely in terms of finding the parameters.

Conversely, the Fixed SIP method proved to be extremely efficient over time with minimal complexity and greater reliability. Since the allocation of money was done monthly regardless of how the market behaved, the investor was consistently exposed to the market. In the event that the market remained positive without any retracement, the strategy of Dynamic SIP underperformed when compared to Fixed SIP because the remaining cash could not be used to purchase stocks as the price kept going up. In terms of investing strategy, Fixed SIP is far more effective as no tuning or market timing was required. This is why the heuristic of constantly investing regardless of market behavior is so popular- because it yields extremely strong returns.

4.3 Trend-following Strategies and Overfitting

In addition, the results indicate that the trend-following strategies underperformed considerably compared to the SIP approaches. The Golden Cross technique yielded an ending portfolio worth about \$252,189, while the 6-Month Momentum strategy finished at around \$223,614. Switches from cash to investments made it difficult for the trend-following strategies to participate in market growth when recovering from crashes. Hence, the results show that trend-following techniques can limit the investor's participation when there are stock market crashes but have a lag in their signals and compounding power.

Another point of discussion is the danger of underfitting or overfitting- the dynamic SIP did perform better than the fixed SIP, but that is simply on the SPX data from 2000-2020. On a different data set, the results could be drastically different. Moreover, from an investing standpoint, it is unimportant if not irrelevant that a model performed well on past data- what truly is significant is how it performs in the real market. That sort of insight can only be gleaned after the fact, after sufficient time as past and the money has already been spent. Because of this, one can also argue that the model.

In summary, it appears that while Dynamic SIP may offer benefits when the market is undergoing bearish conditions or quick recovery periods, the difference from the fixed strategy was not significant enough to justify the extra complexity of the algorithm, namely in finding and tuning the parameters appropriately. The fixed SIP method was the best choice since it was both more practical and yielded almost the same returns as the dynamic SIP method.

4.4 Trading Strategies Compared

Strategy: Fixed SIP

Final Portfolio Value	Total Invested	Total Return	CAGR	Max Drawdown	Total Trades
\$276886.13	\$120000.00	130.74%	4.27%	48.72%	240

Strategy: Dynamic SIP (MA=70/200, Dip=0.96, Exp=1.02, Cap=36)

Final Portfolio Value	Total Invested	Total Return	CAGR	Max Drawdown	Total Trades
\$276909.95	\$120000.00	130.76%	4.27%	42.54%	240

Strategy: Golden Cross (50/200 MA)

Final Portfolio Value	Total Invested	Total Return	CAGR	Max Drawdown	Total Trades
\$252188.92	\$902923.64	-72.07%	-6.18%	16.05%	19

Strategy: 6-Month Momentum

Final Portfolio Value	Total Invested	Total Return	CAGR	Max Drawdown	Total Trades
\$223613.52	\$1764592.59	-87.33%	-9.81%	14.42%	43

4.5 SPX vs NVDA/AMZN

An important element in the study that helped extend the scope involved analyzing the performance of various strategies using different asset classes – the SPX index against high-growth tech stocks such as NVDA (NVIDIA Corp.) and AMZN (Amazon.com Inc.). The comparison demonstrates the importance of asset characteristics in terms of their effect on investment strategy effectiveness.

NVDA and AMZN demonstrated much higher volatility in comparison with SPX, which is seen in more price swings. Such volatility led to both higher gains and higher maximum drawdowns for all analyzed strategies. The largest maximum drawdowns occurred during corrections for NVDA and AMZN and not for the relatively safe and diversified SPX index.

The role of assets' properties in strategy performance is particularly noticeable when talking about sensitivity to certain types of strategies. Momentum and moving average-based strategies worked better on NVDA and AMZN because of the strong positive trend of the two and frequent momentum changes. At the same time, they had more trading signals since moving averages were broken more frequently.

However, contrary to the above, the Fixed SIP approach showed more consistency in performance across all the assets considered, especially in SPX. The investment approach was less vulnerable to volatility and the quick movement of prices due to the periodic nature of investment. Nevertheless, consistency was achieved with low capture of upside performance in comparison to other strategies used for highly volatile assets.

Conclusively, the findings from the analysis indicate that asset selection is crucial in strategy effectiveness. Stable assets like SPX would be better suited to investment strategies such as Fixed SIP that favor stability, whereas highly volatile stocks like NVDA and AMZN would benefit from the strengths and weaknesses of momentum investment strategy.

5. Conclusion

Through this project, it was proven possible to incorporate computer science fundamentals within a realistic financial simulation system. It is evident that the *StockSim* system successfully and fluidly incorporated concepts like object-oriented programming, linked lists, queues, stacks, binary search trees, templates, iterators, and dynamic memory allocation to develop a useful historical market analysis program.

One of the most significant achievements of the entire project is the ability to relate data structures with their respective use. Linked lists could be used for sequential historical prices, while circular queues made it easier to calculate the moving averages. Stacks would support the undo operation, queues managed pending trade operations, and binary search trees made it possible to rank the performance of the portfolio efficiently. This project helped to bridge the gap between theory in the classroom and actual practice in application.

The back-testing results conducted using 2000 – 2020 S&P 500 showed that Dynamic SIP outperformed Fixed SIP in relation to portfolio performance. This happened since Dynamic SIP allowed more investment to be done during market downturns, leading to accumulation of discounted stock prices.

There were some limitations found through the simulation, namely that the model did not consider transaction costs, taxes, dividends, slippage, or liquidity issues. There could be implications for these aspects of trading that could significantly impact the overall performance of the strategy.

Some possible enhancements were considered, including making the system capable of dealing with multi-asset portfolio management, using other sophisticated technical analysis tools, using risk-adjusted position sizing, changing the structures to use balanced search trees (either AVL or Red-Black), adding a GUI, and using real-time market feeds instead of legacy data.

In all, the project offered practical application in software development, algorithmic design, financial modeling, and advanced C++ programming techniques. Furthermore, the project was an opportunity to learn about the applications of theoretical computer science concepts to practical financial problems.